FinResearch AI

Multi-Agent Financial Research System

Complete Project Documentation

+ Demonstration Script

- Updated edition -- includes the new Validation Agent (8 agents total).
- Lists every agent, API, database table, test, and configuration item.
- Ends with a step-by-step demo script covering the website AND the code,
- citing the exact file and function for every logic explanation.

Project Evaluation Edition | Prepared for team demonstration

Includes: Architecture - Agents - Metrics - Reliability - Validation Agent - Demo Script

Contents

1. Problem and Solution
2. Why Multi-Agent Architecture
3. System Architecture
4. All Agents at a Glance (8 agents + orchestrator)
5. Agent Details: What Each Agent Does, File by File
6. How the Agents Interact (Pipeline and Job Lifecycle)
7. How We Measure the Metrics
8. Reliability and Graceful Degradation
9. NEW - Validation Agent Deep Dive
10. Complete Project Inventory (APIs, Tables, Tests, Config)
11. Tech Stack and Deployment

12. DEMO SCRIPT - Website Showcase + Code Walkthrough

- 12.1 Demo Setup Checklist and Timing
- 12.2 Part A - Website Demonstration Steps
- 12.3 Part B - Code Walkthrough (file + what the code does)
- 12.4 Part C - Logic Explained (what to say)
- 12.5 Demo Do-Not List and Q&A Backup

1. Problem and Solution

THE PROBLEM

- Annual and quarterly reports run 100+ pages. Analysts spend hours reading them by hand.
- AI chat tools invent numbers, mix up fiscal years, and show no sources for their claims.
- Small teams and students cannot afford Bloomberg-style terminals or manual deep research.
- Nothing is auditable: when an AI answer looks wrong, there is no way to check where it came from.

HOW WE SOLVE IT

- Upload a filing (PDF or TXT). Eight specialised agents analyse it end to end automatically.
- The Extraction Agent pulls the full financial metric set; every metric stores its source line.
- The Red Flag Agent finds risks in the filing language and always quotes the exact sentence.
- Research answers carry [S1]-style citations you can click and verify. Reports and benchmarks are generated from real extracted data only.
- A new Validation Agent checks that the upload really is a financial document, and rejects resumes, invoices, novels and other wrong files by name.

Mission: trustworthy financial AI -- every number sourced, every claim cited, every input verified.

2. Why Multi-Agent Architecture

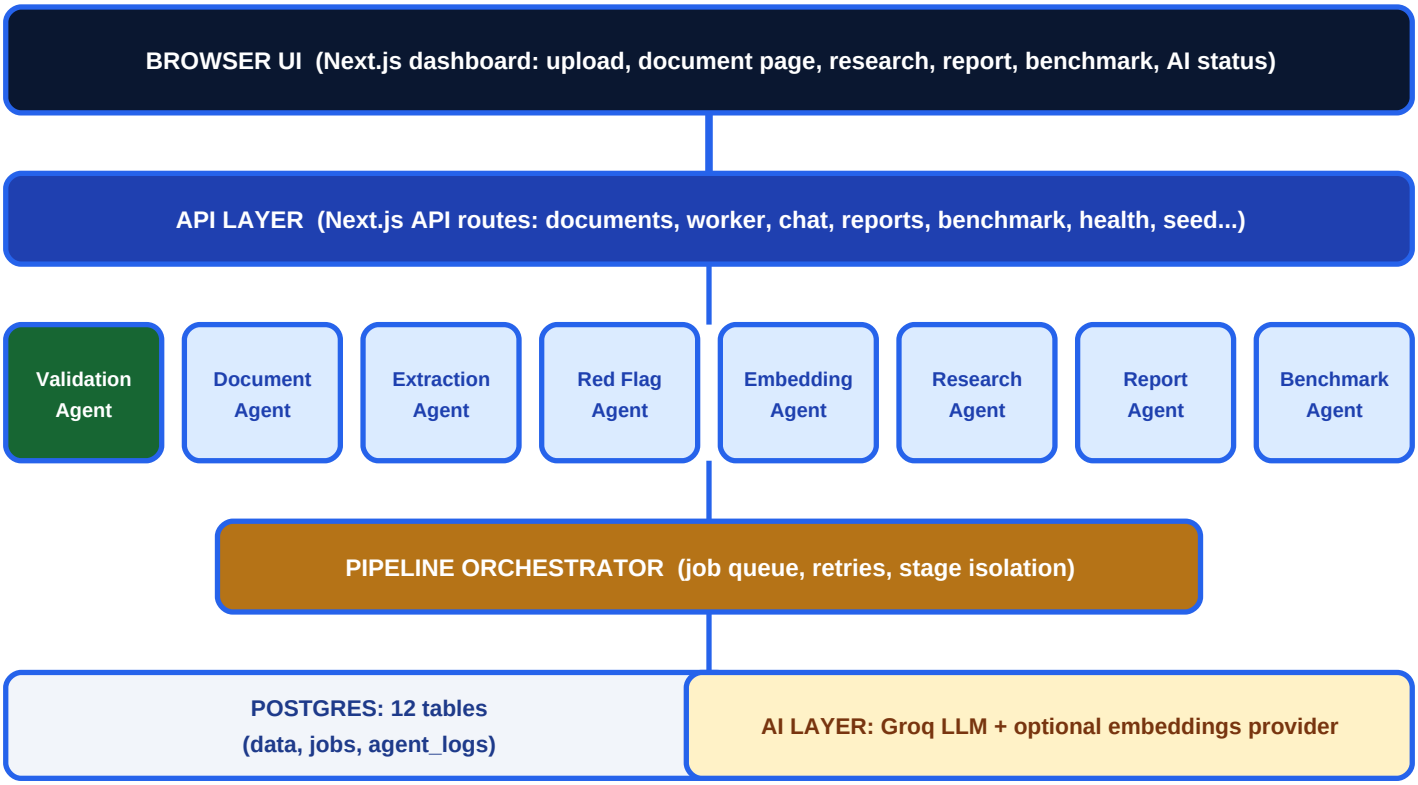
One big AI prompt is a black box. When it fails, everything fails, and nothing can be tested in isolation. We split the job into focused agents instead.

- Specialisation: each agent does one job well -- chunking, extraction, risk scanning, search, writing, comparing, validating.
- Numbers by code, language by AI: deterministic parsers compute the figures; the LLM handles reading and writing.
- Fallbacks everywhere: AI offline -> rule-based mode. Embeddings off -> keyword search. One stage fails -> the rest continue.
- Testable: every agent has its own function surface and unit tests. Failures are isolated and logged to agent_logs.
- Auditable: evidence lines, citations, and agent logs make every output traceable back to the filing.

SINGLE LLM vs MULTI-AGENT

Aspect	Single LLM	Multi-Agent (us)
Numbers	Hallucinated / unverifiable	Computed by code + evidence lines
Sources	None	[S1] citations + exact quotes
Failure	One failure kills the answer	Isolated; status 'partial'
Testing	Impossible to unit-test	Per-agent test suites
Cost	Every call is a big prompt	Small targeted calls + local fallbacks

3. System Architecture



Every stage writes to PostgreSQL (including agent_logs). The AI layer is OPTIONAL: if it is unreachable, deterministic rule-based modes take over and the pipeline keeps working.

- Upload flow: POST /api/documents -> content checks -> VALIDATION AGENT gate -> save + queue job -> orchestrator runs pipeline stages.
- On-demand flows: research questions (chat), report generation, and benchmarking call their agents directly with the saved data.

4. All Agents at a Glance

Eight specialised agents plus one orchestrator. The Validation Agent is the newest addition.

#	Agent	Job in one line	Main file
1	Validation Agent	Is this upload really a financial document?	src/lib/agents/validationAgent.ts
2	Document Agent	Chunk, section, index -- make it searchable	src/lib/agents/documentAgent.ts
3	Extraction Agent	Extract financial metrics + evidence lines	src/lib/agents/extractionAgent.ts
4	Red Flag Agent	Find risks in filing language, with quotes	src/lib/agents/riskAgent.ts
5	Embedding Agent	Semantic search vectors (optional)	src/lib/agents/embeddingAgent.ts
6	Research Agent	Answer questions with [S1] citations	src/lib/agents/researchAgent.ts
7	Report Agent	Full analysis report from real data	src/lib/agents/reportAgent.ts
8	Benchmark Agent	Compare companies / periods safely	src/lib/agents/benchmarkAgent.ts
--	Pipeline Orchestrator	Queue jobs, retry, isolate failures	src/lib/agents/orchestrator.ts

Agents 1-5 run automatically on every upload. Agents 6-8 answer on demand. The orchestrator runs the automatic pipeline and never trusts the AI blindly -- helpers in extractionUtils.ts and analysisUtils.ts verify every number and every piece of evidence.

5. Agent Details (File by File)

1. Validation Agent

`src/lib/agents/validationAgent.ts`

validateFinancialDocument(content, fileName) scores the text: report sections +3 each, metric-label lines +2 (capped), finance-vocabulary density (capped, scaled by length), $/% figures, period markers. Wrong-type detectors (resume, invoice, recipe, book, academic paper, source code, slides) fire only for weak documents and reject with the TYPE NAMED. logDocumentValidation() writes the result to agent_logs. Optional AI tie-breaker for borderline scores.

2. Document Agent

`src/lib/agents/documentAgent.ts`

chunkText(text, 1500, 200) splits text into overlapping chunks. extractSections() detects report sections. processDocument(documentId, content) stores chunks into document_chunks. searchDocumentChunks / searchDocumentCollection power retrieval.

3. Extraction Agent

`src/lib/agents/extractionAgent.ts`

extractFinancialMetrics() asks the AI for structured JSON metrics, then extractionUtils.ts takes over: extractMetricsLocally() parses statement labels deterministically, evidenceSupportsMetricValue() checks the number against the evidence line, and mergeWithLocalMetrics() makes the LOCAL number win whenever it conflicts with the AI.

4. Red Flag Agent

`src/lib/agents/riskAgent.ts`

scanForRisks() combines two detectors from analysisUtils.ts: detectTextualRedFlags() (filing-language risk phrases with severity) and detectFinancialTrendRisks() (computed from metrics). Every flag is grounded with hasGroundedEvidence() and carries an exact quote from the filing.

5. Embedding Agent

`src/lib/agents/embeddingAgent.ts`

createTextEmbedding() / indexDocumentEmbeddings() build semantic vectors when an embeddings provider is configured (OpenAI-compatible: Ollama locally, Gemini or Nomic on Render). tryCreateQueryEmbedding() falls back gracefully -- if disabled, search uses keywords and the status shows 'Disabled', never an error.

5. Agent Details (continued)

6. Research Agent

`src/lib/agents/researchAgent.ts`

answerResearchQuestion() is not allowed to invent: it first retrieves real text (searchDocumentChunks / embeddings), then answers ONLY from that context, tagging every statement with [S1], [S2]... style citations that point to the retrieved source chunks.

7. Report Agent

`src/lib/agents/reportAgent.ts`

generateAnalysisReport() builds the full report: executive summary, company overview, financial analysis (from the extracted metrics table), risk analysis (from risk_flags), outlook, and a sources list. Tables are rendered from numbers by code; the narrative is written by AI strictly from those numbers.

8. Benchmark Agent

`src/lib/agents/benchmarkAgent.ts`

generateBenchmarkInsights() compares companies/periods with hard guards: exact metric values are packed into the prompt (so the AI cannot round or invent), incompatible periods are flagged before comparing, and the verdict is written only from verified fields.

Pipeline Orchestrator

`src/lib/agents/orchestrator.ts`

runDocumentPipeline() chains the automatic stages. processDocumentJob(jobId) claims one job atomically (UPDATE ... WHERE status='queued'), runs stages with per-stage try/catch so one failure cannot kill the rest, and writes progress to agent_logs. processDueDocumentJobs() is the batch entry point used by the worker route.

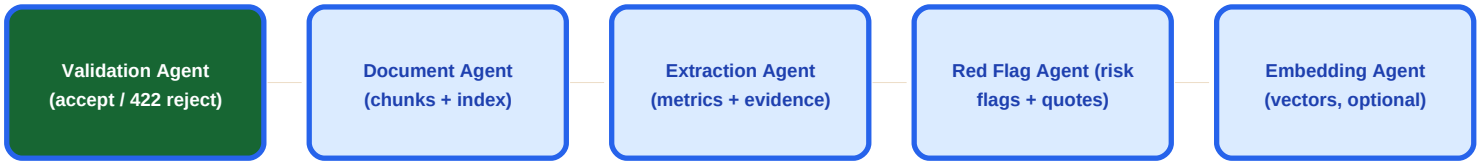
Shared AI + Utils

`src/lib/agents/llmClient.ts + src/lib/agents/aiStatus.ts`
`src/lib/agents/extractionUtils.ts + src/lib/agents/analysisUtils.ts`

llmClient.ts: generateJsonLlm / generateWithLlm with retry backoff (computeRetryDelayMs). aiStatus.ts: isLlmConfigured / isEmbeddingConfigured -- the AI is always optional. extractionUtils + analysisUtils: every deterministic number and evidence check used to keep the AI honest.

6. How the Agents Interact

UPLOAD PIPELINE (automatic)



JOB LIFECYCLE (src/lib/agents/orchestrator.ts + src/app/api/worker/route.ts)

- Upload creates a row in document_processing_jobs with status 'queued'.
- The worker route (protected by CRON_SECRET) claims jobs in batches and calls processDocumentJob(jobId).
- The claim is ATOMIC: UPDATE ... WHERE status = 'queued' -- two workers can never run the same job.
- Each stage runs inside its own try/catch: a failing stage is logged in agent_logs and marked, while the remaining stages still run.
- Stale locks (older than 15 minutes) are reclaimed automatically, so a crashed job never blocks the queue.

ON-DEMAND AGENTS

- Research (chat): retrieve real chunks -> answerResearchQuestion() with [S1] citations.
- Report: generateAnalysisReport() over the saved metrics + flags -> analysis_reports.
- Benchmark: generateBenchmarkInsights() across companies -> benchmark_comparisons.

7. How We Measure the Metrics

The Extraction Agent measures the full financial metric set per document (stored in the financial_metrics table). All figures come from the filing itself -- never from the AI's memory.

Category	What is measured	How it is measured
Scale & growth	Revenue, revenue growth	Statement-label parser (regex) + AI JSON
Profitability	Operating income, net income, margins	extractMetricsLocally() wins conflicts
Per-share	EPS, share counts	Exact label match (e.g. 'Earnings per share')
Balance sheet	Assets, liabilities, equity, debt, cash	Deterministic parsing of statement lines
Cash flow	Operating cash flow, capex, free cash flow	Evidence line saved with each metric
Expenses	R&D, SG&A and other cost lines	Evidence-support check before saving

- Every metric row stores: value + the exact evidence line from the filing (findMetricEvidenceLine in analysisUtils.ts).
- mergeWithLocalMetrics(): when the AI and the deterministic parser disagree, the LOCAL (code-computed) value wins.
- evidenceSupportsMetricValue(): a metric is saved only if its evidence line actually supports the number.
- Growth rates, margins, and ratios are computed by code from the stored numbers -- not asked from the AI.
- Quality gate: research answers and report prose must match source text (hasGroundedEvidence) before being shown.

8. Reliability and Graceful Degradation

If this happens...	The system does this	Where
AI provider offline / no key	Deterministic rule-based mode; status shows 'local'	aiStatus.ts + extractionUtils.ts
Embeddings not configured	Keyword search retrieval; status 'Disabled'	embeddingAgent.ts
One pipeline stage crashes	Stage marked failed; remaining stages still run	orchestrator.ts
Wrong file type uploaded	422 with the detected type named (e.g. 'Resume/CV')	validationAgent.ts + documents/route.ts
Corrupt / scanned PDF	Clear 'PDF extraction failed' error, not a stack trace	documents/route.ts
Job queue crash / restart	Stale locks reclaimed after 15 minutes	orchestrator.ts
Bad AI JSON output	parseJsonResponse fallback + local merge; nothing saved wrong	llmClient.ts + extractionUtils.ts

- Security: hashed passwords (NextAuth + bcrypt), protected routes, CRON_SECRET for the worker, SEED_SECRET for seeding, upload size caps, and per-user data scoping.
- Everything is logged: every agent run writes to agent_logs (agent name, action, status) -- visible on the document page's Agent Processing Summary.
- Cost: the entire stack runs on free tiers -- $0 infrastructure (Vercel/Render free, Groq free, optional free embeddings).

9. NEW - Validation Agent Deep Dive

THE QUESTION IT ANSWERS: is the file the user uploads really a financial document? Wrong files are rejected at upload with the detected type named -- before any processing time is wasted.

SCORING (validateFinancialDocument in src/lib/agents/validationAgent.ts)

Signal	Points	Notes
Report section headings (annual report title, MD&A, financial statements, notes, auditor's report...)	+3 each	Line-anchored patterns
Metric-label lines like 'Total revenue: $383,285'	+2 each	Capped at 14
Finance vocabulary density	up to +10	Scaled by document length
Currency / percentage figures	up to +4	Real filings are figure-heavy
Period markers ('fiscal year ended', quarters)	+2	Annual reports always have these
Wrong-type detector fires (resume, invoice, recipe, book, academic paper, source code, slides)	-100	Only on weak documents; the TYPE IS NAMED

- ACCEPT only when structure is real (2+ sections or 4+ metric labels) AND score >= 8. High confidence at score >= 15 with 3+ sections.
- Borderline weak documents (score 4-12) get ONE optional AI tie-breaker on the first 1,500 characters -- wrapped in try/catch so the AI can never crash an upload.
- On reject: HTTP 422 with 'This file does not look like a financial document (looks like: Resume/CV)' plus the top reasons -- shown as the red toast.
- On accept: logDocumentValidation() writes 'Validation Agent - Financial document type check' into agent_logs, shown as the green card in the Agent Processing Summary.
- 10/10 sandbox tests pass (4 real annual reports accepted; resume, invoice, novel, recipe, academic paper, cover letter, news rejected). On the live project the self-test passes 3/3.

Files: src/lib/agents/validationAgent.ts (the agent) + src/app/api/documents/route.ts (where it is called).

10. Complete Project Inventory

EVERYTHING IN THE PROJECT -- the full list

Area	Items
Agents	8 agents + Pipeline Orchestrator (Validation, Document, Extraction, Red Flag, Embedding, Research, Report, Benchmark)
Agent files	src/lib/agents/: validationAgent, documentAgent, extractionAgent, riskAgent, embeddingAgent, researchAgent, reportAgent, benchmarkAgent, orchestrator, llmClient, aiStatus, extractionUtils, analysisUtils
UI pages	Dashboard, upload, document detail (metrics + risks + agent cards), research/chat, reports, benchmark, AI status
API routes	health, auth, documents (+ id, embeddings, reprocess), worker, dashboard, companies, metrics, risks, reports, chat, benchmark, sessions, seed
Database (12 tables)	users, research_sessions, companies, documents, document_chunks, document_processing_jobs, financial_metrics, risk_flags, analysis_reports, chat_messages, benchmark_comparisons, agent_logs
Unit tests	npm test suites: analysisUtils.test.ts, aiStatus.test.ts, extractionUtils.test.ts, llmClient.test.ts (plus test-validation-agent.cjs self-test)
Demo data	demo-test-data/: 28 PDFs (Apple, Microsoft, Amazon, Google, Meta, NVIDIA, Infosys x FY2023-2026) + fake-resume.txt
Config / env vars	DATABASE_URL, DIRECT_URL, NEXTAUTH_SECRET, NEXTAUTH_URL, CRON_SECRET, SEED_SECRET, GROQ_API_KEY (optional), EMBEDDING_BASE_URL / EMBEDDING_API_KEY / EMBEDDING_MODEL (optional trio)

11. Tech Stack and Deployment

Layer	Choice
Framework	Next.js 16 (App Router) + TypeScript + Tailwind CSS
Database	Neon PostgreSQL + Drizzle ORM (serverless driver)
Auth	NextAuth (credentials, bcrypt-hashed passwords)
AI	Groq (Llama / GPT-OSS class models) via OpenAI-compatible API -- OPTIONAL
Embeddings	Any OpenAI-compatible provider (Ollama local / Gemini or Nomic on Render) -- OPTIONAL
PDF parsing	pdf-parse (with clear 422 errors for corrupt files)
Jobs	PostgreSQL job queue with atomic claim + worker route (CRON_SECRET)
Deployment	Render (free tier) + GitHub auto-deploy; UptimeRobot keeps it awake

- Build on Render: npm ci && npm run build. Start: npm start. Node 22 (.nvmrc).
- Embedding config needs ALL THREE of EMBEDDING_BASE_URL + EMBEDDING_API_KEY + EMBEDDING_MODEL -- or the system runs in keyword mode.
- Health check: GET /api/health (public) reports AI + embeddings status and the agent list.
- Cost to run: $0 (free tiers throughout). Cold start ~50 seconds; UptimeRobot pings every 10 minutes.

12. DEMO SCRIPT - Website + Code

PURPOSE: show the website working, then show the code behind it, and explain the logic of each key file. Every code claim below cites the exact file and function.

12.1 DEMO SETUP CHECKLIST

- Project running: npm run dev (local) or the live Render URL. Logged in.
- Files ready on the desktop: fake-resume.txt and one demo PDF (e.g. Apple FY2024 annual report from demo-test-data/).
- Browser tabs prepared: (1) the app, (2) GET /api/health, (3) your code editor at the project root.
- Optional: code editor open at src/lib/agents/ so files are one click away.

DEMO FLOW (recommended order)

Step	Show	Time
A1	Health check -- system + 8 agents listed	20 sec
A2	Upload the WRONG file (fake-resume.txt) -> red rejection toast	30 sec
A3	Upload the RIGHT file (Apple PDF) -> 6 agent cards incl. Validation	45 sec
A4	Metrics table + Red Flag risks with exact quotes	30 sec
A5	Research question -> answer with [S1] citations	30 sec
A6	Report generation + Download TXT	25 sec
A7	Benchmark AAPL vs MSFT	20 sec
B*	Code walkthrough (files on the next pages)	90 sec

12.2 Part A - Website Demonstration

A1. Health check

SHOW: Open GET /api/health in the browser.

Point at the agents array (8 agents incl. Validation) and the ai/Embeddings status. Say: 'Everything is self-reporting -- the system tells you what is working.'

A2. Reject a wrong file

SHOW: Upload page -> choose fake-resume.txt -> Upload.

The red toast appears: 'This file does not look like a financial document (looks like: Resume/CV)'. Say: 'The new Validation Agent blocks wrong uploads and names the type it detected.'

A3. Upload a real filing

SHOW: Upload the Apple FY2024 demo PDF (any company name/ticker).

Open the document: the Agent Processing Summary shows 6 cards -- the green Validation card reads 'Confirmed: Annual report'. Say: 'Accepted with confidence, then Document, Extraction, Red Flag and Embedding agents follow.'

A4. Metrics + risks

SHOW: Scroll to the metrics table and risk flags.

Point at one metric value and its evidence line. Point at one risk and read its exact quote. Say: 'Every number has a source line; every risk has a quote. Nothing is invented.'

A5. Research with citations

SHOW: Ask: 'What was the revenue and what are the key risks?'

The answer shows [S1]-style tags. Say: 'The Research Agent may only answer from retrieved text -- and cites it.'

A6. Report

SHOW: Click Generate Report -> Tables + Narrative tabs -> Download TXT.

Say: 'Tables are rendered by code from real numbers; the narrative is written by AI from those numbers only.'

A7. Benchmark

SHOW: Compare AAPL vs MSFT.

Say: 'The Benchmark Agent refuses to compare incompatible periods and packs exact numbers into the prompt so the AI cannot alter them.'

12.3 Part B - Code Walkthrough (File -> What the Code

Open each file in the editor while speaking. Say what the code does, then show the key function.

File	What the code does	Point at / say
src/lib/agents/ validationAgent.ts	Verifies the upload is a financial document; scores sections, metric labels, finance vocabulary; names wrong types (resume, invoice...).	validateFinancialDocument() + logDocumentValidation() -- 'the gatekeeper we added'. Rejects with 422 + reasons.
src/app/api/documents/ route.ts	The upload endpoint: size/content checks, THEN the validation gate, then pdf-parse extraction and the job queue.	'Right here the file is checked -- if it is a resume it stops with 422 before any processing.'
src/lib/agents/ documentAgent.ts	Splits text into chunks and sections and stores them for search.	chunkText() 1500/200 overlap -- 'the basis of all retrieval'. processDocument() writes document_chunks.
src/lib/agents/ extractionAgent.ts	Extracts the financial metric set as structured JSON from the report.	extractFinancialMetrics() -- 'the AI proposes, the code disposes' (see next row).
src/lib/agents/ extractionUtils.ts	Deterministic metric parsing + verification; LOCAL values win over the AI.	extractMetricsLocally(), evidenceSupportsMetricValue(), mergeWithLocalMetrics() -- 'numbers are never blindly trusted to the AI'.
src/lib/agents/ riskAgent.ts	Finds risk signals in filing language and computed trends.	scanForRisks() -- combines detectTextualRedFlags() + detectFinancialTrendRisks() from analysisUtils.ts; every flag has an exact quote.
src/lib/agents/ analysisUtils.ts	Grounding + evidence helpers used by every agent.	hasGroundedEvidence(), findMetricEvidenceLine() -- 'the honesty layer of the project'.
src/lib/agents/ embeddingAgent.ts	Semantic vectors, optional; falls back to keyword search.	createTextEmbedding() / tryCreateQueryEmbedding() -- 'if embeddings are off, nothing breaks'.
src/lib/agents/ researchAgent.ts	Answers questions ONLY from retrieved text with citations.	answerResearchQuestion() -- '[S1] tags make every claim verifiable'.
src/lib/agents/ reportAgent.ts	Builds the full analysis report from saved metrics + flags.	generateAnalysisReport() -- 'tables by code, narrative by AI from real data'.
src/lib/agents/ benchmarkAgent.ts	Compares companies with hard guards.	generateBenchmarkInsights() -- 'exact numbers packed in; incompatible periods refused'.
src/lib/agents/ orchestrator.ts	Runs the pipeline: atomic job claim, retries, per-stage isolation.	processDocumentJob() -- 'one stage down, the rest continue'; 15-min stale-lock reclaim.
src/app/api/worker/ route.ts	Claims queued jobs in batches (protected by CRON_SECRET).	'The engine that keeps the queue moving without duplicates'.

File	What the code does	Point at / say
src/lib/agents/ llmClient.ts	All AI calls + JSON parsing + retry backoff.	generateJsonLlm(), computeRetryDelayMs() -- 'resilient AI access'.
src/lib/agents/ aiStatus.ts	Reports + enforces optional-AI design.	isLlmConfigured(), isEmbeddingConfigured() -- 'the system works with AI off'.
src/db/schema.ts	The 12 tables incl. agent_logs (every agent action is recorded).	'Auditability by design -- the activity trail comes from agent_logs'.

12.4 Part C - Logic Explained (What to Say)

Validation logic

`src/lib/agents/validationAgent.ts`

'Real annual reports have structure: section titles, metric labels like TOTAL REVENUE: value, lots of figures, period language. Weak documents don't. Our scorer rewards structure and, for weak documents, the wrong-type detectors name exactly what the file looks like. Borderline cases get one optional AI tie-break that can never crash the upload.'

Honest numbers

`src/lib/agents/extractionUtils.ts`

'The AI proposes metrics as JSON, but a deterministic parser re-reads the statements. If they disagree, the code wins. A metric is saved only when its evidence line actually supports the number.'

Grounded risks

`src/lib/agents/riskAgent.ts + analysisUtils.ts`

'Risk flags come from filing language and computed trends -- never from AI imagination. Each flag stores the exact sentence from the filing.'

Cited answers

`src/lib/agents/researchAgent.ts`

'The Research Agent retrieves real chunks first, then answers only from them, tagging [S1] [S2]. If retrieval finds nothing, it says so instead of inventing.'

Resilience

`src/lib/agents/orchestrator.ts + aiStatus.ts`

'Jobs are claimed atomically, stages are isolated, stale locks self-heal, and the AI layer is optional at every step. The system degrades gracefully instead of breaking.'

The big idea

`all of the above`

'Numbers by code. Language by AI. Trust by evidence.'

12.5 Demo Do-Not List + Q&A Backup

DO NOT (common demo mistakes)

- Do not upload a wrong file expecting success -- show the rejection ON PURPOSE as a feature.
- Do not claim 'PDF export' -- reports download as .TXT (the numbers are in the tables).
- Do not show the embeddings step as 'failed' -- if the provider is off, the card says 'Disabled (optional)' and search still works.
- Do not edit code live during the demo -- have the files open and just walk through them.

Q&A BACKUP (hard questions, honest answers)

Question	Answer
What if the AI invents a number?	Code verification wins (mergeWithLocalMetrics) and evidence lines are stored; answers are checked with hasGroundedEvidence.
What if there is no AI key at all?	The system runs in deterministic mode: local extraction, risk rules, keyword search. AI is optional by design (aiStatus.ts).
Why multi-agent instead of one prompt?	Isolation, fallbacks, and testability -- see Section 2. One failing stage can never kill the pipeline.
How do we know it is a real report?	The Validation Agent scores structure and names wrong types; 10/10 automated tests plus the live self-test pass.
Is it expensive to run?	No -- $0: free-tier hosting, free Groq AI, optional free embeddings.
Where is the activity trail?	agent_logs table, rendered as the Agent Processing Summary on each document page.

CLOSING LINE

FinResearch AI: trustworthy financial research -- numbers by code, language by AI, trust by evidence. Uploaded files verified. Every claim cited. $0 to run.